Grounding DINO Reproduction and Evaluation

Student Name: 王扬皓, 张泰玮, 郑翔越
Student ID: 12310802, 12310801, 12310805

Student Name: 王扬皓, 张泰玮, 郑翔越

Student ID: 12310802, 12310801, 12310805

1 1. Introduction

Open-vocabulary object detection aims to localize objects described by arbitrary text instead of a fixed category set. Visual grounding is a related task where a model localizes the image region described by a natural-language expression, such as ”the red car on the left” or ”the person holding an umbrella”. These tasks are more challenging than classical closed-set recognition because they require both visual recognition and language understanding.

This project reproduces and evaluates Grounding DINO for open-vocabulary object detection (OVD) and visual grounding (VG). We use the official Grounding DINO Swin-T OGC checkpoint and evaluate it in a zero-shot setting without COCO or RefCOCO fine-tuning. The goal is to build a reproducible evaluation pipeline, compare results with the original paper, and analyze where the reproduction succeeds or differs.

The project covers three main parts: method reproduction, dataset evaluation, and experimental analysis. For OVD, we evaluate on COCO val2017. For VG, we evaluate on RefCOCO, RefCOCO+, and RefCOCOg. The final self-implemented OVD pipeline achieves **46.16 mAP** on COCO val2017, and the official Grounding DINO COCO script reaches **48.50 mAP**. For VG, our reproduced zero-shot results match or slightly exceed the paper’s zero-shot @1 results on five evaluated splits.

2 2. Related works

DETR introduced an end-to-end transformer formulation for object detection. DINO improves DETR-style detection with denoising anchor boxes and stronger query initialization. Grounding DINO builds on this detection framework and extends it to language-conditioned detection.

Grounding DINO combines a Swin Transformer visual backbone with a BERT text encoder and cross-modal fusion modules. Instead of predicting only a fixed set of classes, it uses text prompts to guide detection and can localize objects described by natural language. This makes it suitable for both OVD and VG.

Other related open-vocabulary or vision-language detection systems include GLIP, OWL-ViT, YOLO-World, and Detic. We choose Grounding DINO because it directly supports both open-set detection and referring expression grounding, and because official checkpoints and evaluation scripts are publicly available.

The datasets used in this project are also standard benchmarks in this area. COCO is a widely used object detection benchmark with official mAP evaluation through COCO API / pycocotools. RefCOCO, RefCOCO+, and RefCOCOg evaluate visual grounding on natural-language referring expressions built on COCO images.

3 3. Method

We reproduce an existing method rather than proposing a new model. The reproduced model is Grounding DINO with the official Swin-T OGC checkpoint `groundingdino_swint_ogc.pth`.

The model pipeline is:

1. Encode the image with the Swin-T visual backbone.
2. Encode the text prompt or referring expression with BERT.
3. Fuse visual and text features with the Grounding DINO cross-modal transformer.
4. Decode candidate boxes and token-aligned phrase scores.
5. Convert model outputs into task-specific predictions.

For OVD, the input text is a prompt containing the 80 COCO class names. We use a `concat_token` prompt mode, which concatenates class names and maps model token spans back to COCO category IDs. The final predictions are evaluated with the COCO API.

For VG, the input text is a referring expression. The model returns multiple candidate boxes, and we select the best candidate using a semantic matching strategy. A prediction is counted as correct if the selected box has IoU at least 0.5 with the ground-truth box.

The implementation supports two model backends:

Backend	Role	Final usage
`gdino`	Official GroundingDINO source + OGC checkpoint	Main result backend
`hf`	HuggingFace `IDEA-Research/grounding-dino-tiny`	Fallback and early validation

Important implementation files include:

- `src/model_wrapper.py`: unified model loading and inference.
- `src/ovd/eval_coco.py`: COCO OVD evaluation.
- `scripts/run_vg_eval.py`: RefCOCO-family VG evaluation.
- `src/ovd/official_postprocess.py`: OVD token-span mapping.
- `src/vg/box_selection.py`: VG box selection strategies.

4 4. Experiments

4.1 4.1 Datasets

We evaluate on one object detection dataset and three visual grounding datasets.

Task	Dataset	Images	Annotation	Scale
OVD	COCO val2017	`data/coco/val2017`	`instances_val2017.json`	5000 images
VG	RefCOCO	COCO train2014	RefCOCO annotations / HF parquet	val, testB
VG	RefCOCO+	COCO train2014	RefCOCO+ annotations / HF parquet	val, testB
VG	RefCOCOg	COCO train2014	RefCOCOg annotations / HF parquet	val

This project uses zero-shot inference. We do not train or fine-tune on RefCOCO. Therefore, the VG results should be compared to the Grounding DINO paper's zero-shot @1 results rather than the much higher fine-tuned RefCOCO results.

4.2 4.2 Implementation Details

The main implementation uses PyTorch and the official GroundingDINO source backend. The model checkpoint is `weights/groundingdino_swint_ogc.pth`.

Important settings:

Item	Setting
Model	Grounding DINO Swin-T OGC
Framework	PyTorch
OVD prompt mode	`concat_token`
OVD prompt template	`{name}`
OVD best threshold	box=0.15, text=0.05
VG selection	`semantic`
VG threshold	box=0.05, text=0.05

The OVD threshold was selected through threshold sweep. The VG threshold was selected through a small validation subset sweep, where box=0.05 and text=0.05 achieved the best Acc@0.5.

We also validated the GroundingDINO CUDA extension. On an RTX 5090 server, the original extension failed to compile under PyTorch 2.8 because it used older PyTorch C++ APIs such as `tensor.type()` and `tensor.data<T>()`. After patching those calls to PyTorch 2.x compatible APIs, `groundingdino._C` imported successfully. The CUDA extension improved inference speed but did not materially change OVD mAP, so the final accuracy discussion focuses on evaluation protocol and post-processing.

4.3 4.3 Metrics

For OVD, we use COCO mAP computed by COCO API / pycocotools. The main metric is mAP averaged over IoU thresholds from 0.50 to 0.95. We also report AP50, AP75, AP for small/medium/large objects, and average recall.

For VG, we use Acc@0.5. A prediction is correct if the selected predicted box has IoU at least 0.5 with the ground-truth box. This corresponds to the paper's zero-shot @1 metric.

4.4 4.4 Experimental design & results

4.4.1 OVD on COCO val2017

The main OVD result is from `supplementary/metrics/ovd_self_metrics.json`, with the corresponding raw predictions saved in `supplementary/predictions/ovd_self_predictions.json`.

Metric	Result
mAP	**46.16**
AP50	61.20
AP75	50.39
AP_s	31.26
AP_m	49.08
AP_l	60.72
AR@1	35.88
AR@10	58.39
AR@100	61.88

Comparison with the paper and the official script:

Method	COCO val2017 mAP	Notes
Grounding DINO paper	48.4	Reported zero-shot result
Official `test_ap_on_coco.py`	**48.50**	Reproduced in this project
Our self-implemented pipeline	**46.16**	Best full-run result after threshold sweep

The official script result confirms that the checkpoint and dataset are correct. The remaining gap in our own pipeline is most likely caused by post-processing details, token-to-category mapping, and evaluation protocol differences.

4.4.2 Visual grounding on RefCOCO-family datasets

The main VG results are from `supplementary/metrics/vg_full_metrics.json`, with the corresponding raw predictions saved in `supplementary/predictions/`.

Dataset / split	Our Acc@0.5	Hits / Total	Paper zero-shot @1	Difference
RefCOCO val	**50.85%**	5509 / 10834	50.41%	+0.44
RefCOCO testB	**45.38%**	2312 / 5095	43.21%	+2.17
RefCOCO+ val	**51.67%**	5559 / 10758	51.40%	+0.27
RefCOCO+ testB	**46.45%**	2271 / 4889	45.81%	+0.64
RefCOCOg val	**60.95%**	2984 / 4896	60.42%	+0.53

These results show that the reproduced VG pipeline is aligned with the paper's zero-shot setting. All five evaluated splits match or slightly exceed the reported zero-shot @1 numbers.

4.4.3 Prompt ablation

Prompt ablation results are from `supplementary/metrics/prompt_ablation.json`.

The results are:

- Prompt template `{name}`: mAP **47.05**, AP50 **58.80**, 2731 predictions.
- Prompt template `a {name}`: mAP 17.56, AP50 24.21, 1675 predictions.
- Prompt template `a photo of a {name}`: mAP 5.24, AP50 6.33, 185 predictions.

Direct class names work best because COCO OVD concatenates 80 category names into a single prompt. Longer templates consume the BERT token budget and can cause later categories to be truncated or poorly aligned.

4.4.4 VG threshold and protocol checks

The best VG threshold on a 500-sample subset is:

box_threshold	text_threshold	Acc@0.5	Samples
0.05	0.05	**52.20%**	500

We also compare two VG box selection strategies:

Strategy	Acc@1	Acc@5	Acc@10
semantic	52.20%	89.60%	95.60%
argmax_official	**52.60%**	89.60%	95.60%

The top-1 difference is only 0.4 percentage points, and top-5 / top-10 are identical. This suggests that candidate box quality is more important than this specific selection rule.

4.4.5 Engineering validation

The RTX 5090 CUDA extension validation gives:

Experiment	mAP	Speed observation	Role
5090 fallback	44.60	About 8.8 images/s	Engineering check
5090 CUDA fixed	44.60	About 10.7 images/s	Engineering check

The CUDA extension improves speed but does not explain the OVD accuracy gap. Therefore, the main gap between 46.16 and 48.50 is more likely due to post-processing and protocol mismatch.

4.4.6 Limitations

The OVD self-implemented pipeline is still below the official script. Since the official script reaches 48.50 mAP with the same checkpoint and dataset, the gap is likely not caused by model weights or data preparation.

Small objects remain difficult: AP_s is 31.26, while AP_m is 49.08 and AP_l is 60.72. This is expected because small objects have weaker visual features after downsampling and are harder to align with text queries.

For VG, this project only evaluates zero-shot inference. It does not reproduce the paper's fine-tuned RefCOCO results in the 81-89% range. Reproducing those numbers would require supervised training on RefCOCO splits and additional compute resources.

If we continued the project, the next steps would be to fully match the official COCO post-processing in our own OVD pipeline, evaluate on LVIS or ODinW, and implement RefCOCO fine-tuning.

5 5. Conclusion

This project tackles open-vocabulary object detection and visual grounding using Grounding DINO. We reproduce the official model pipeline, evaluate it on COCO val2017 and RefCOCO-family datasets, and analyze prompt design, thresholds, selection protocol, CUDA extension behavior, and failure modes.

The OVD self-implemented pipeline reaches **46.16 mAP** on COCO val2017, while the official COCO evaluation script reaches **48.50 mAP**. The visual grounding pipeline reaches **50.85%**, **45.38%**, **51.67%**, **46.45%**, and **60.95%** Acc@0.5 on the five evaluated RefCOCO-family splits, matching or slightly exceeding the paper's zero-shot results.

Overall, the project completes the intended reproduction and evaluation work. The main limitation is that our own OVD post-processing does not fully match the official evaluation script. However, the experiments clearly identify this gap and show that the model, datasets, and core evaluation pipeline are working correctly.

6 Reference

1. Shilong Liu et al. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." ECCV 2024.
2. Hao Zhang et al. "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection." ICLR 2023.
3. Tsung-Yi Lin et al. "Microsoft COCO: Common Objects in Context." ECCV 2014.
4. Licheng Yu et al. "Modeling Context in Referring Expressions." ECCV 2016.
5. COCO API: https://github.com/cocodataset/cocoapi
6. pycocotools: https://github.com/ppwwyyxx/cocoapi
7. GroundingDINO official repository: https://github.com/IDEA-Research/GroundingDINO
8. GLIP repository: https://github.com/microsoft/GLIP
9. OWL-ViT documentation: https://huggingface.co/docs/transformers/model_doc/owlvit
10. YOLO-World repository: https://github.com/AILab-CVC/YOLO-World
11. Detic repository: https://github.com/facebookresearch/Detic

7 Contributions

王扬皓 (12310802): 33.33%

张泰玮 (12310801): 33.33%

郑翔越 (12310805): 33.33%

8 GitHub Repository

https://github.com/HaoHaoLucas/cv-project